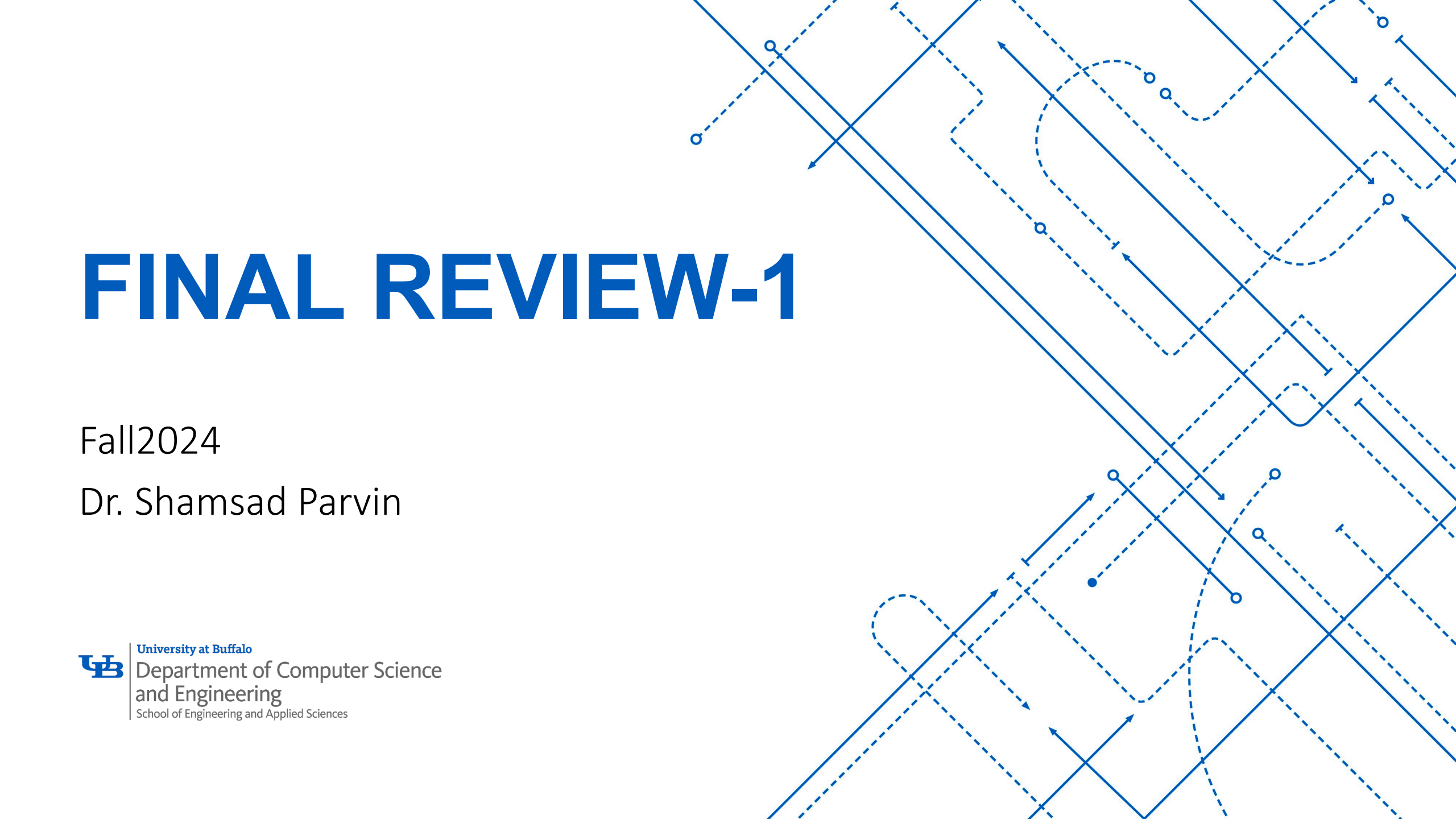


The background of the slide features a complex network of blue lines and arrows. Some lines are solid, while others are dashed. The arrows point in various directions, creating a sense of movement and connectivity. The lines and arrows are primarily concentrated on the right side of the slide, with some extending towards the left.

FINAL REVIEW-1

Fall2024

Dr. Shamsad Parvin

Final Exam Logistics

You should expect 3 to 4 questions

Exam Date: **13th December, Friday**

Total Duration : **2 hrs**

Location: **NSC 201**

Time: **11.00am -2.00pm**

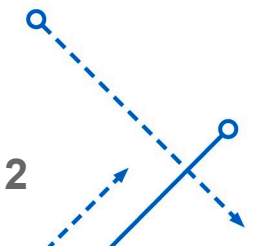
Paper and pen-based exam (closed book)

Your Seat will be randomized

All bags/electronics must be placed at the front of the room

- **What to Bring:**

- Pen/Pencil
- UB ID Card



Final Exam syllabus

Indexing and Hashing

- B tree
- Static and dynamic hashing

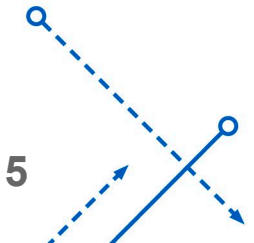
Transactions

Concurrency Control Protocols

Lock based protocol

Timestamp protocol, OCC

Recovery Algorithm (Undo/Redo)



INDEXING, HASHING AND B-TREE

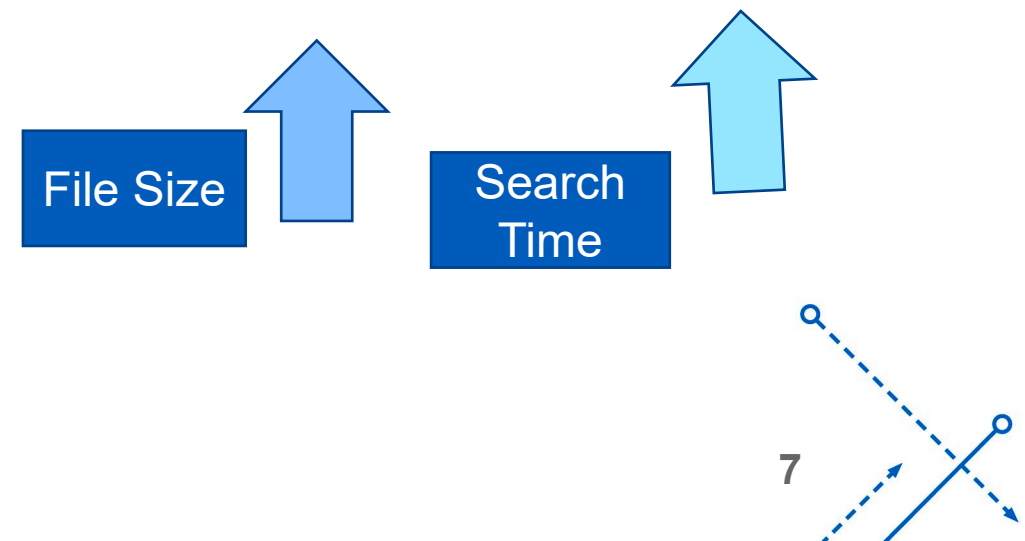
Lec 17-23

Basic Concepts of Indexing

- Indexing mechanisms used to speed up access to desired data.
 - E.g., author catalog in library
 - When the relations are very large: it became expensive to scan all the tuples to find the tuples that matches a given conditions.
 - Example :

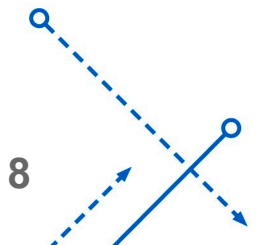
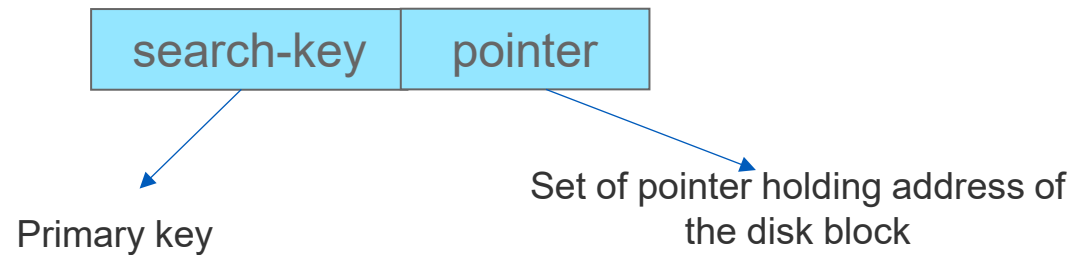
```
SELECT *
FROM Movies WHERE
  studioname="Disney" AND Year =1990
```

Assume that Movies relations has 10,000 tuples where only 100 movies made in year 1990 .



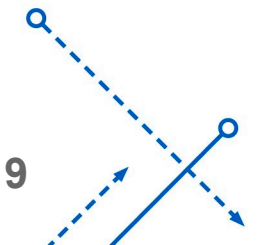
Understand the Basic Concepts of Indexing

- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

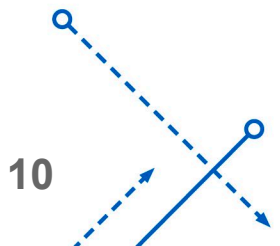


Index Evaluation Metrics

- Many Types of indexes evaluate them on :
- Access types
- Access time
- Insertion time
- Deletion time
- Space overhead



- A **dense index** is a sequence of blocks holding only the keys of the records and pointers to the records themselves
- Index blocks of the dense index maintain these keys in the same sorted order as in the main data file itself.
- Then, by using the index, we can find any record given its search key, with only one disk I/O per lookup.



Dense Index Files : Example

- **Dense index** — Index record appears for every search-key value in the file.
- E.g. index on *ID* attribute of *instructor* relation

10101	→	10101	Srinivasan	Comp. Sci.	65000	
12121	→	12121	Wu	Finance	90000	
15151	→	15151	Mozart	Music	40000	
22222	→	22222	Einstein	Physics	95000	
32343	→	32343	El Said	History	60000	
33456	→	33456	Gold	Physics	87000	
45565	→	45565	Katz	Comp. Sci.	75000	
58583	→	58583	Califieri	History	62000	
76543	→	76543	Singh	Finance	80000	
76766	→	76766	Crick	Biology	72000	
83821	→	83821	Brandt	Comp. Sci.	92000	
98345	→	98345	Kim	Elec. Eng.	80000	

Index file

No. of Row in Index
table= No. of row in the
main table

Sparse Index

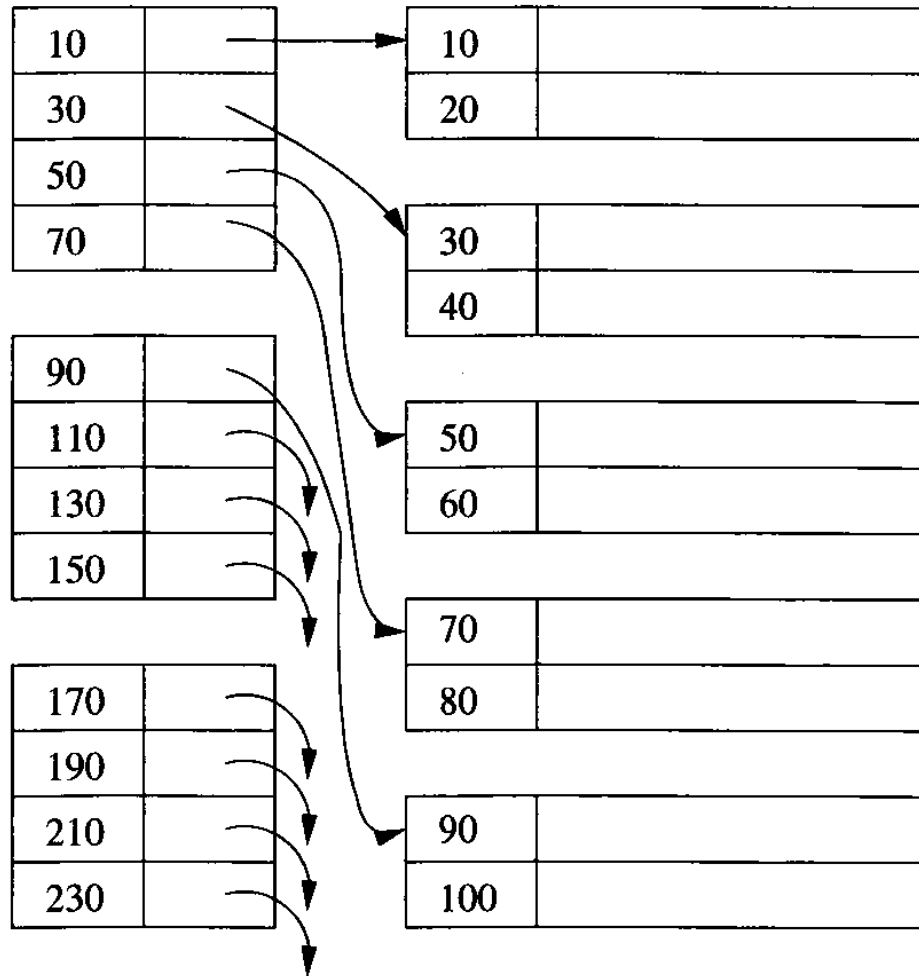
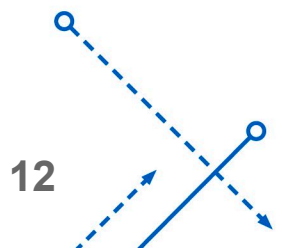


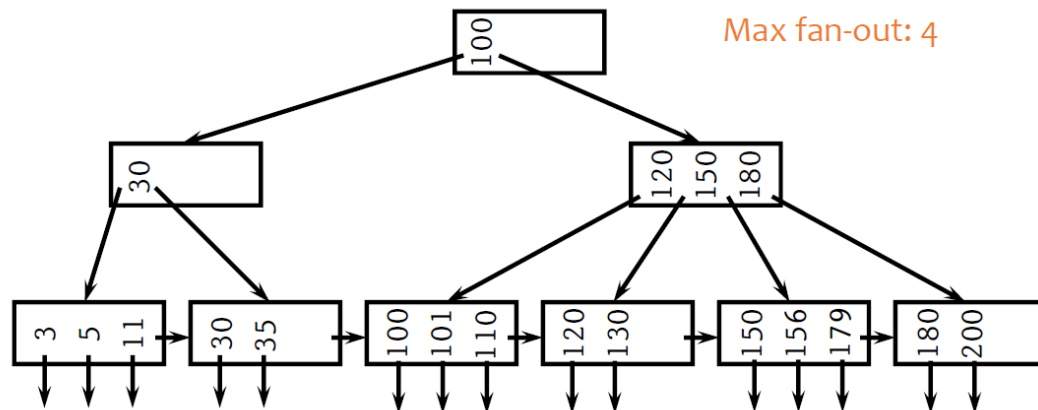
Figure 14.3: A sparse index on a sequential file

- Typically, only one key per data block
- Searching: Find the index record **with largest value that is less or equal to the value we are looking**
- What is its advantage over dense index?**
- It thus uses less space than a dense index, at the expense of somewhat more time to find a record given its key.
- When should you use it?**
- only use a sparse index if the data file is sorted by the search key, while a dense index can be used for any search key.**

No. of Row in Index
table= No. of blocks in
the main table



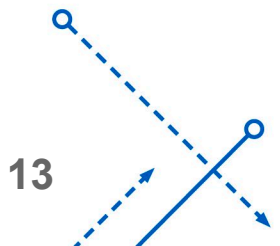
B Trees



P_n = pointer , K_n = search-key



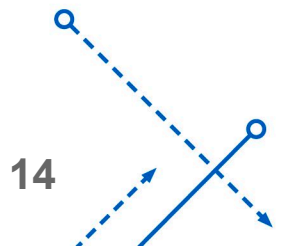
- A more general structure that is commonly used in commercial systems
- The particular variant that is most often used is known as a *B+ tree*
 - B-trees **automatically maintain as many levels of index** as is appropriate for the size of the file being indexed.
 - B-trees manage the space on the blocks they use so that **every block is between half used and completely full**
 - **Balanced (more or less)**, i.e. all the paths from root to leaves have nearly the same length
 - **Disk-based**: one node per block; Each block will have space for n search-key values and $n + 1$ pointers.



Balancing Constraints

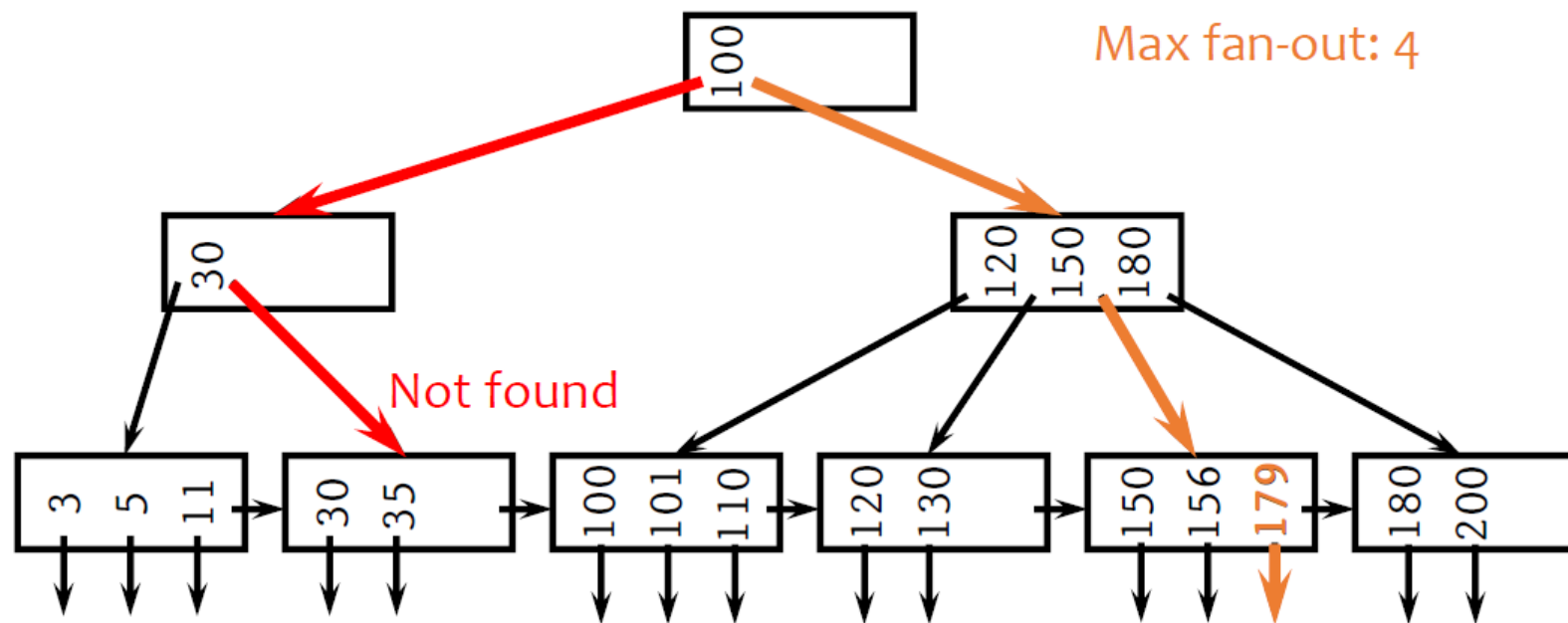
- **Height Constraint:** all leaves at the same lowest level
- **Fan out Constraint:** all nodes at least half full (except root)

	Max # pointers	Max # keys	Min # active pointers	Min # keys
Non-leaf	f	$f - 1$	$\lfloor f/2 \rfloor$	$\lfloor f/2 \rfloor - 1$
Root	f	$f - 1$	2	1
Leaf	f	$f - 1$	$\lfloor f/2 \rfloor$	$\lfloor f/2 \rfloor$



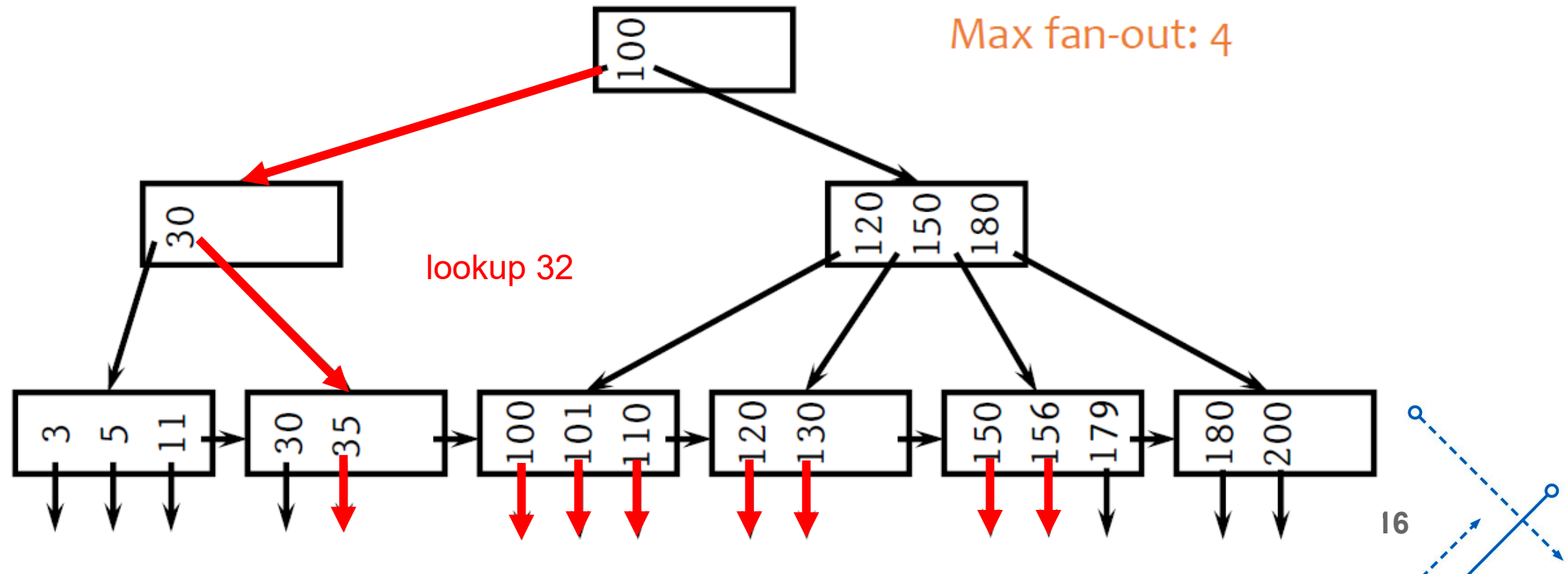
Lookups

- **select * from R where $k == 179$;**
- **select * from R where $k = 32$;**



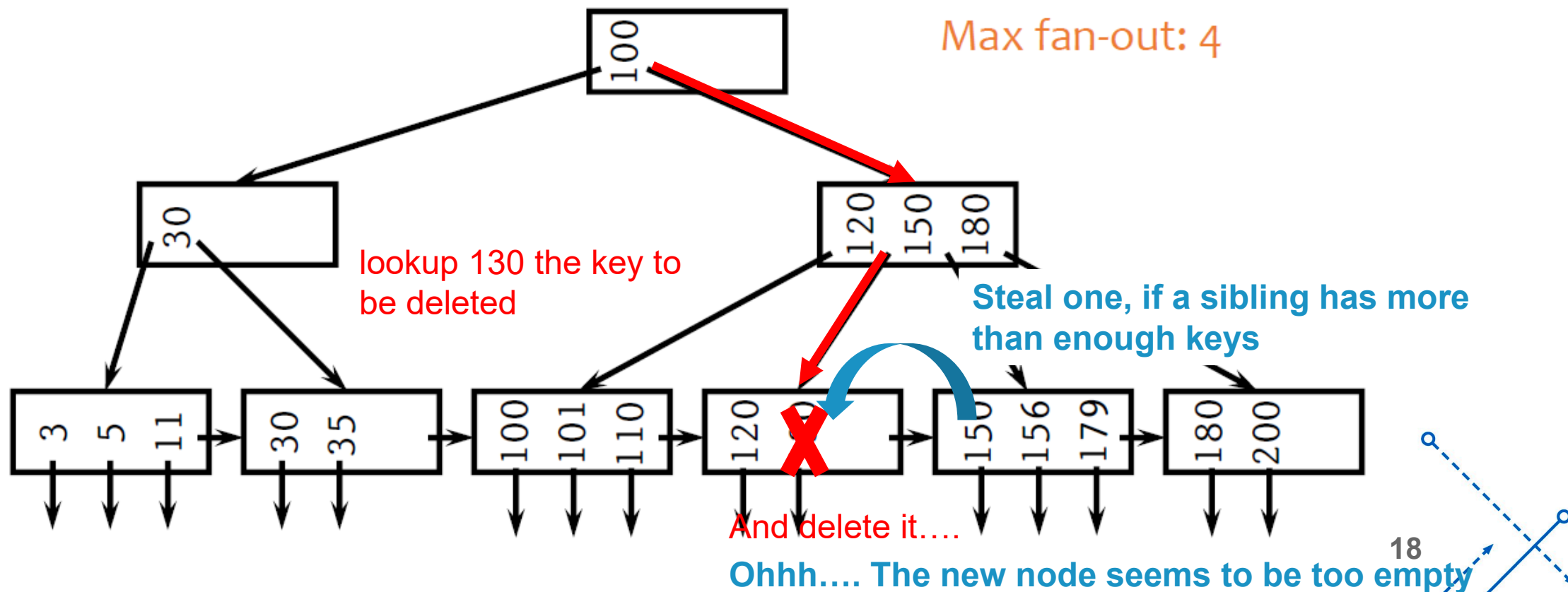
Range Query

- select * from R where $k > 32$ AND $k < 179$;

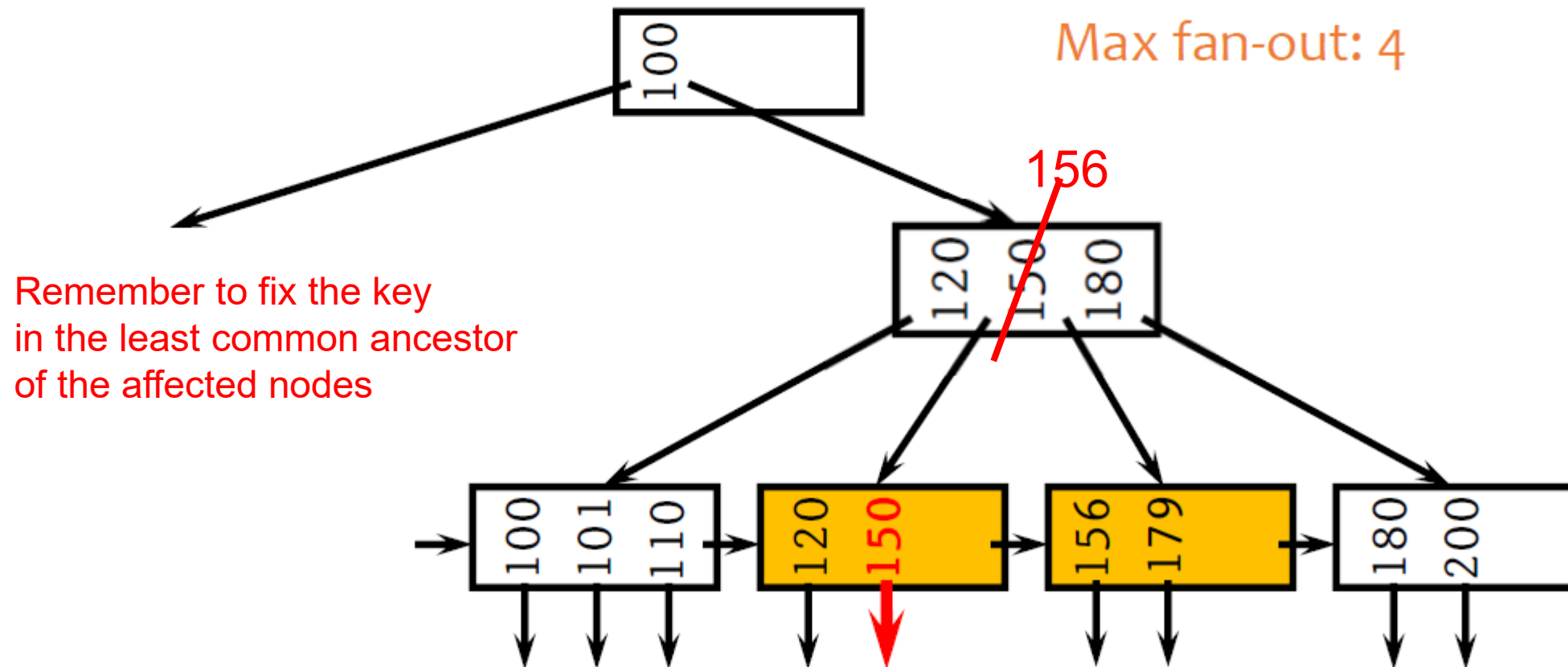


Deletion

- Delete a record with search key value 130

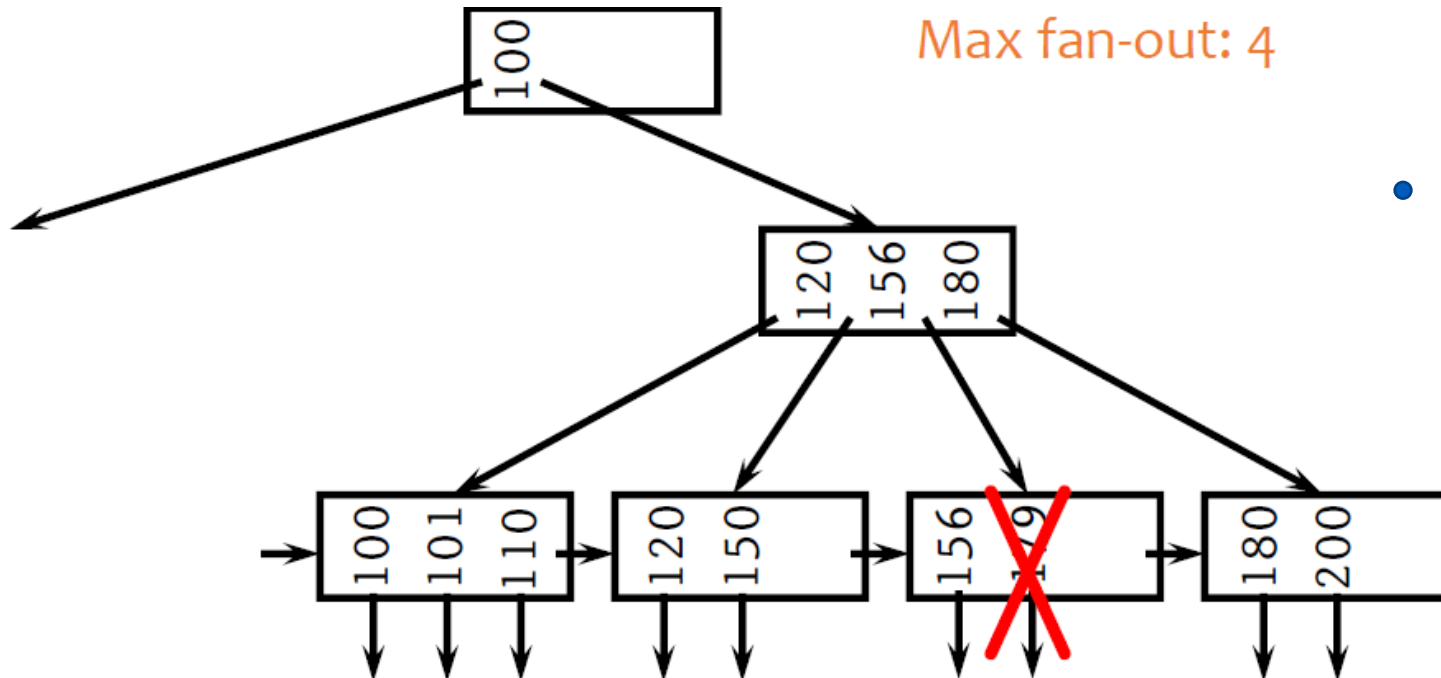


Deletion



Another Deletion Scenario

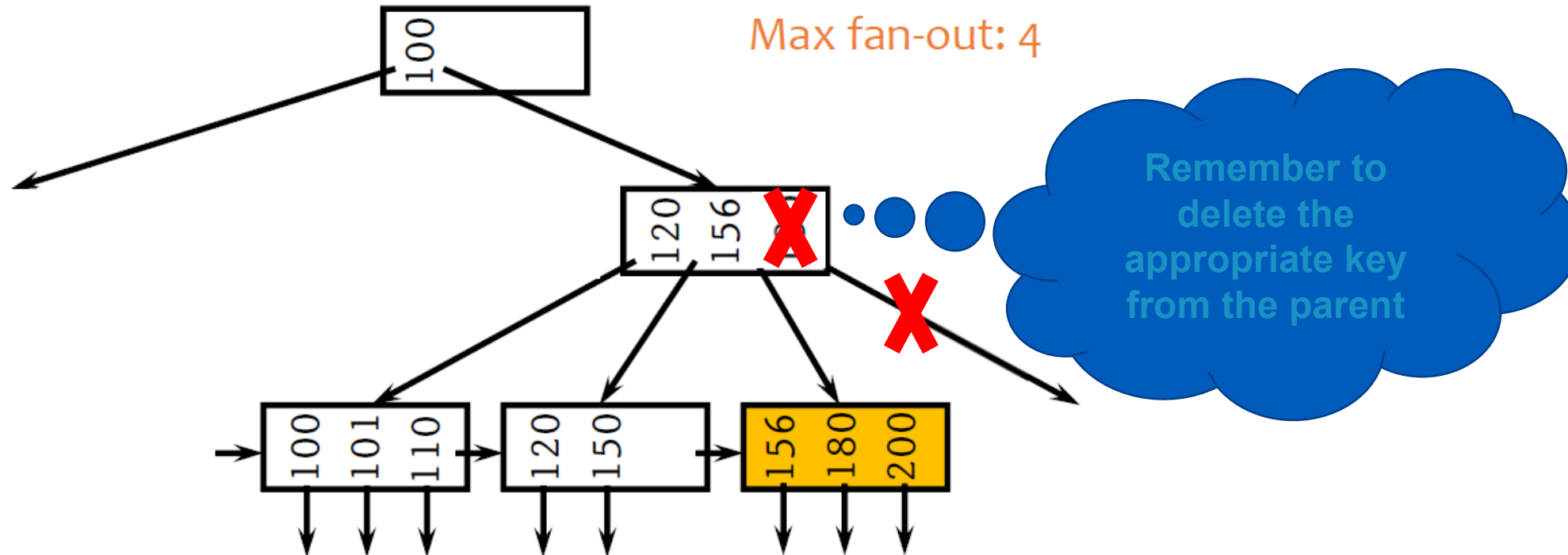
Delete a record with search key value 179



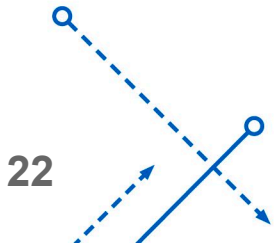
Cannot steal
from siblings!!
Merge then...

Another Deletion Scenario

Delete a record with search key value 179

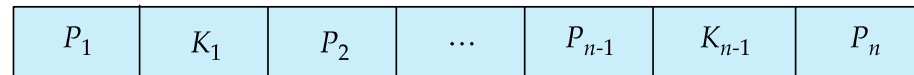


B-Tree



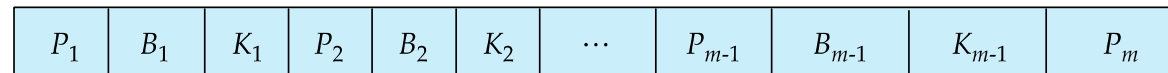
B-Tree Index Files

- B-tree is Similar to B+-tree, but B-tree allows search-key values to appear only once; eliminates redundant storage of search keys from B+-tree.
- Search keys in nonleaf nodes appear nowhere else in the B-tree; an additional pointer field for each search key in a nonleaf node must be included.
- Generalized B-tree leaf node – P_n are the data or record pointer

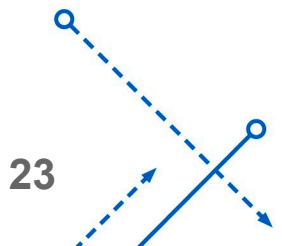


(a)

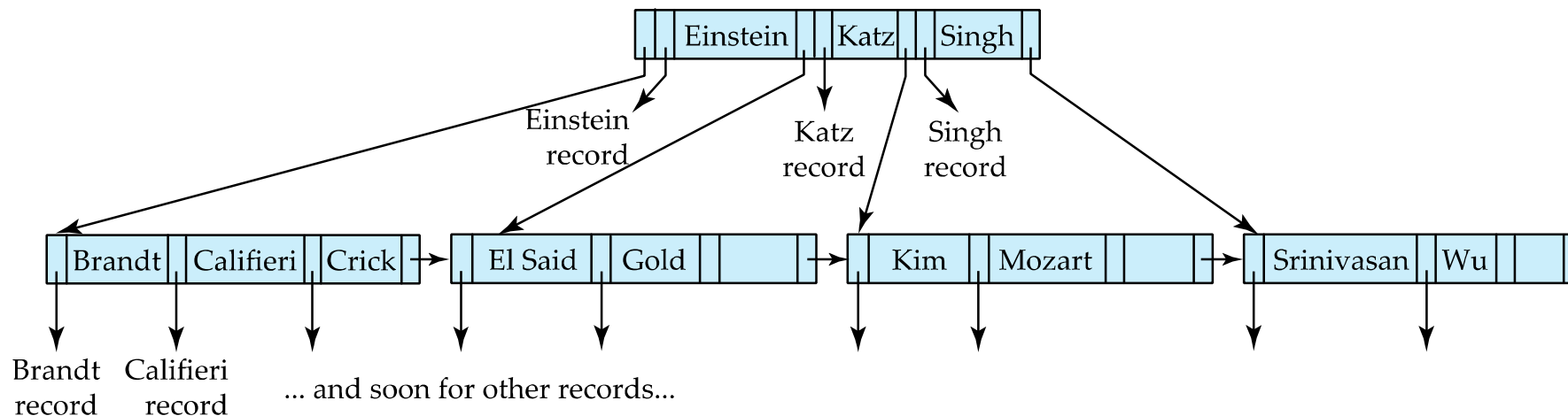
- Nonleaf node – pointers B_m are the bucket or tree pointers.



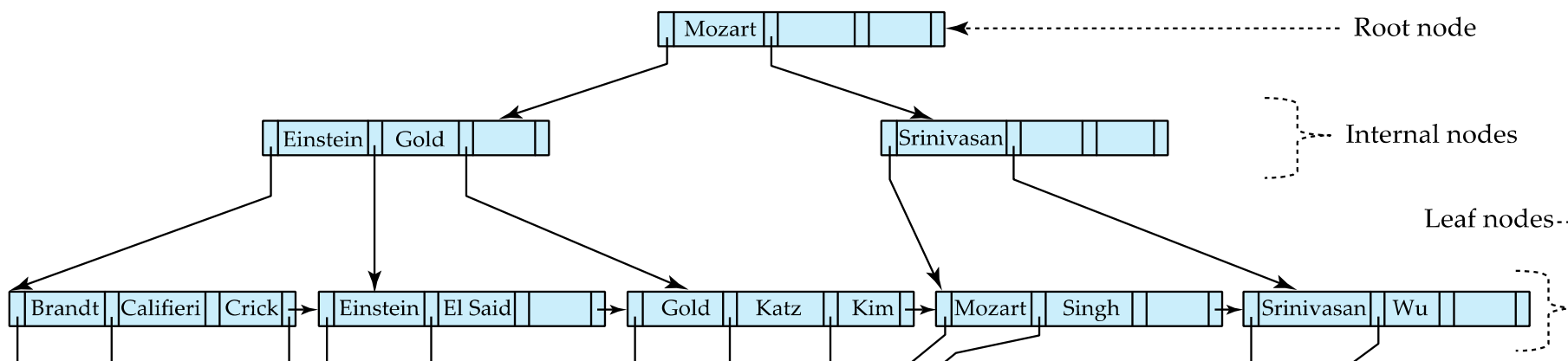
(b)



B-Tree Index File Example



B-tree (above) and B+-tree (below) on same data



B-tree Vs B+ Tree

- We will discuss B+ tree, which has several advantages over standard B tree, which include:
 - Though duplication of keys are maintained, B+ tree **allows the data pointers to be present only in the leaf nodes**, which makes the search and updates more efficient
 - **Leaf nodes are stored as a structurally linked list**
 - **Searching is slower in B-tree compared to B+-tree**
 - **Deletion in B-tree is complicated compared to B+-tree**

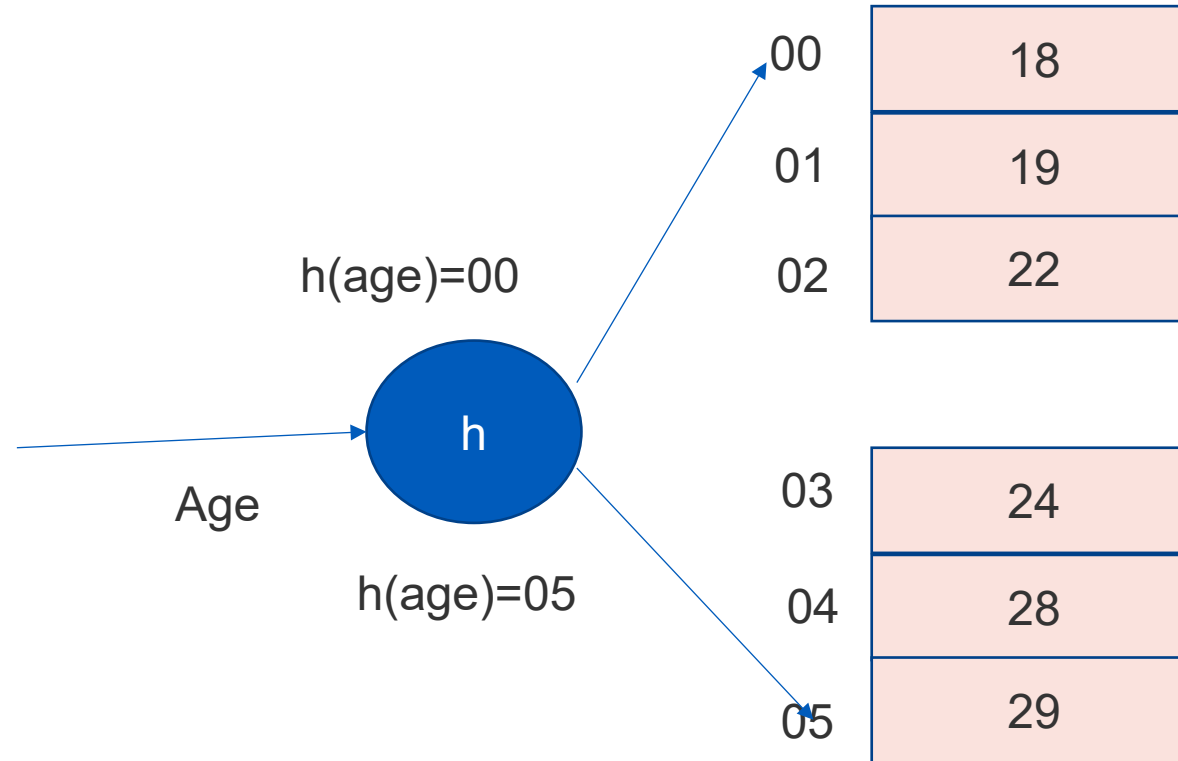


Hash Based Index

- **Records are grouped into buckets**
 - Bucket = primary page plus zero or more overflow pages
- **Hashing function h :**
- A mapping function that maps all the set of search-keys r to the address where actual records are stored. It is a function from search keys to bucket addresses.
 - $h(r)$ = bucket in which (data entry for) record r belongs
 - h looks at the search key fields of r
 - No need for “index entries” in this scheme



Hash Based Index



Types of Hashing

- Static Hashing
- Dynamic Hashing
 - Extensible Hashing
 - Linear Hashing

Understand the basic concepts and differences between the Linear and Extensible hashing



Example-1 : Static Hashing

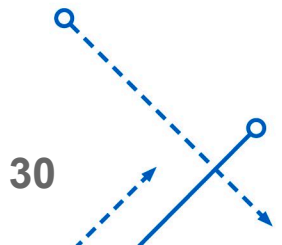
- Insert 5,4,7 and 10 in a Static Hash file.
- What is the Hash function, $h(\text{key})$?
- $h(\text{key}) = \text{Key} \bmod N = \text{Key} \bmod 4$
- If we want to add a record with search key 20
- **Where to place it?**



Types of Hashing

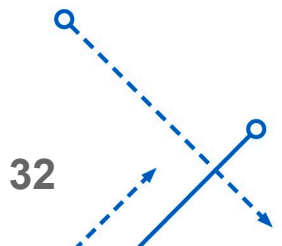
- Static Hashing
- Dynamic Hashing
 - Extensible Hashing
 - Linear Hashing

When does bucket split cause Directory doubling?



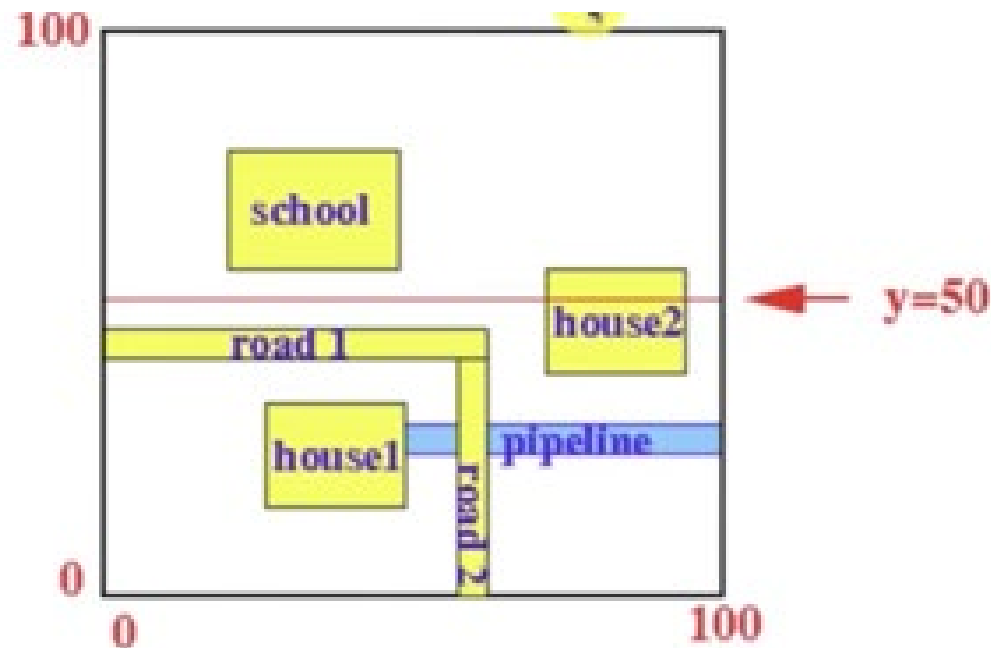
MULTIDIMENSIONAL INDEX

- Until now, we discussed about one-dimensional index structures
- A multi-dimensional index may have several dimensions
- A multi-dimensional index combines several dimensions into one index
 - Although the search key may involve several attributes, the **one-dimensional nature of indexes such as B-trees comes from the fact that values must be provided for all attributes of the search key, or the index is useless.**
 - Hash tables require that the search key be completely known for any lookup. If a key consists of several fields, and even one is unknown, we cannot apply the hash function, but must instead search all the buckets.
 - **Example:** *A geographic information system*



Example Queries

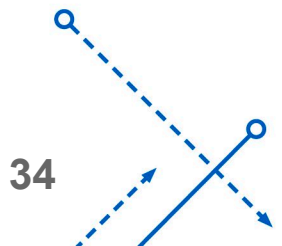
- Lists the example queries where we can use Multi-dimension Indexing
- Partial match queries: Specify values for one or more dimensions and look for all points matching those values in those dimensions.
- Range queries: Give ranges for one or more of the dimensions, and ask for the set of points within those ranges.
- Nearest Neighbor Queries
- Where-am-I queries



Multi-dimensional index structure

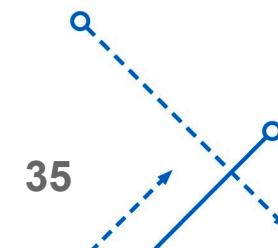
- Two categories
 1. Table based (Hash like) structure
 - Grid File
 - Partitioned Hashing
 2. Tree like (Tree based Structure)
 - Multiple Key index
 - Kd-trees
 - Quad trees
 - R-trees

Understand the concepts of Grid File and Partitioned Hashing



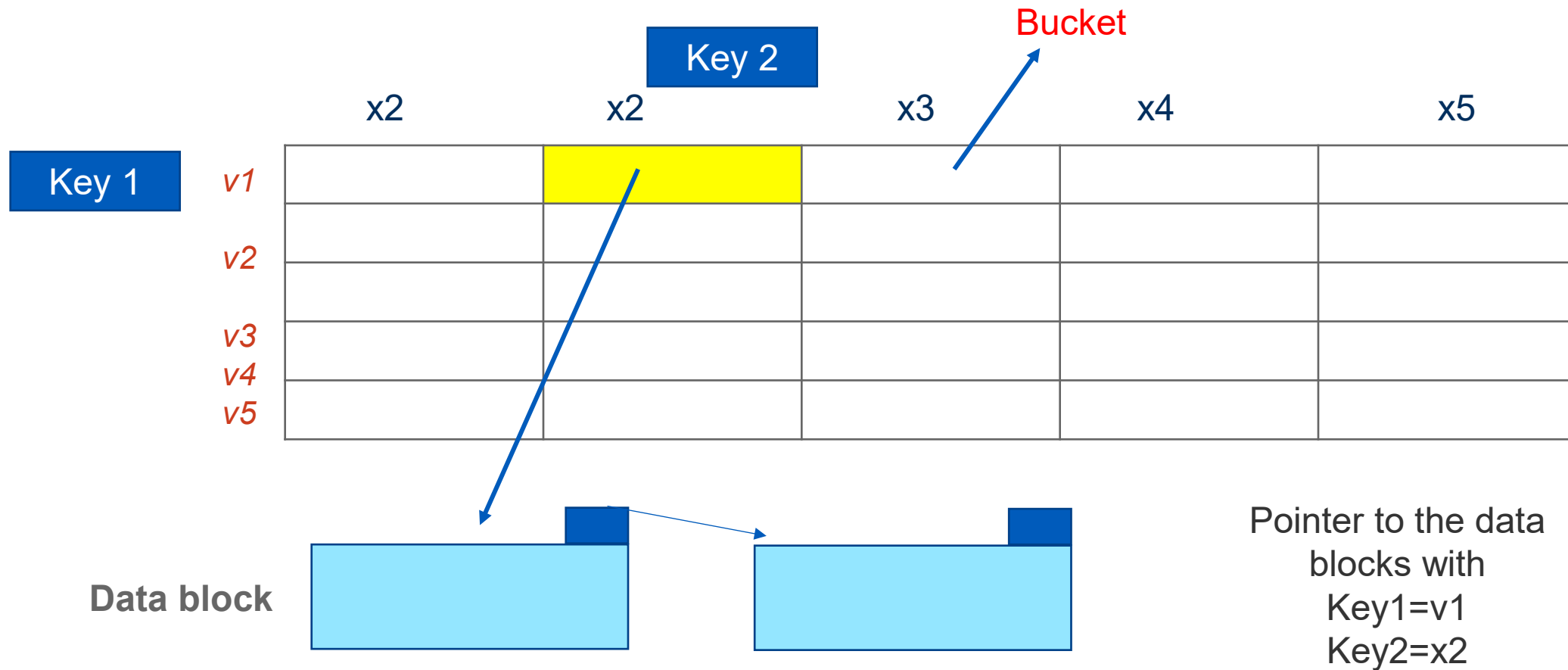
Grid Index

- Partitioned multi-dimensional space with a grid
- Grid Files extend traditional hashing indexes
 - --usually does not have hash values, but regions $h(\text{key}) = \langle x, y \rangle$
 - -- *grid lines* partition the space into **stripes**
 - --Intersections of stripes from different dimensions define regions
- Spacing between adjacent grid lines may also vary
- Each region corresponds to a bucket
- Attribute values for record determine region and therefore bucket.



Grid Index

- Grid Index file : an index that is organized into a t-dimensional structure



Range Interpretation

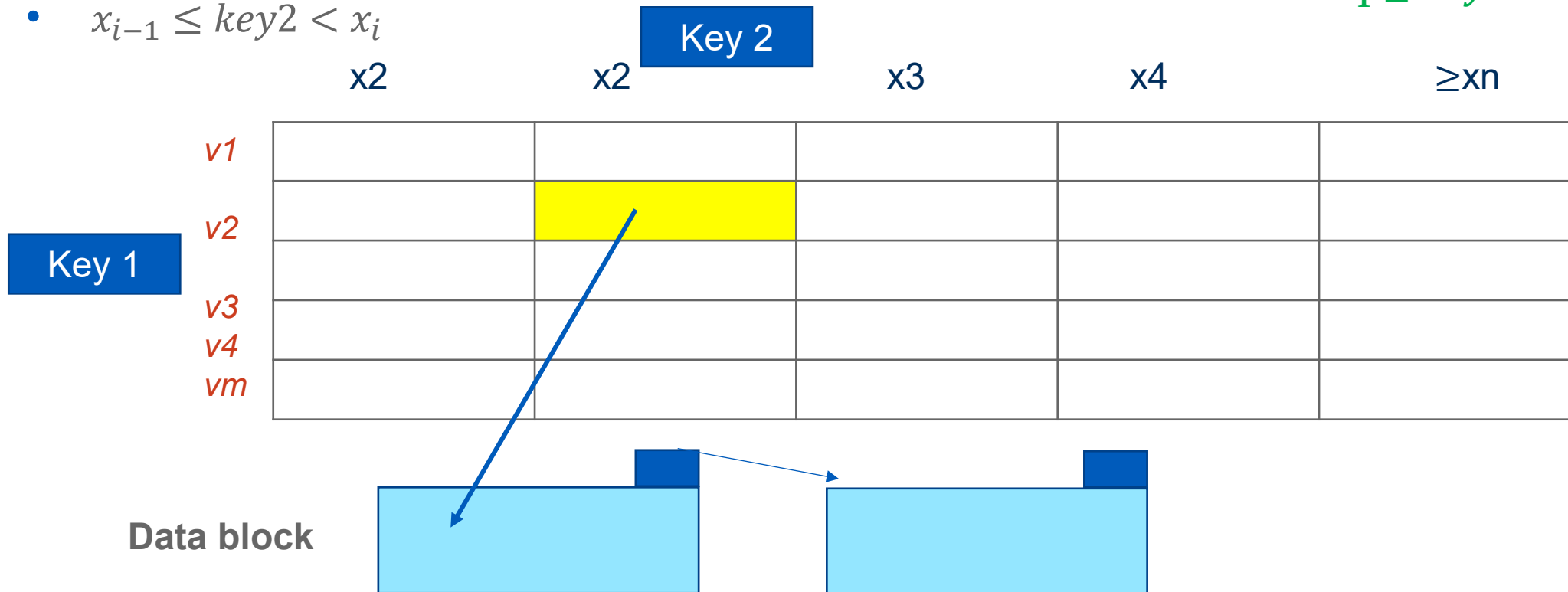
- The bucket pointer in bucket (v_i, x_i) points to data blocks that store search key values:

- $v_{i-1} \leq \text{key1} < v_i$
- $x_{i-1} \leq \text{key2} < x_i$

Pointer to the data blocks with

$$v_1 \leq \text{key1} < v_2$$

$$x_1 \leq \text{key2} < x_2$$



The grid lines represents end points of the range with n grid lines you will have $n+1$ ranges

Example

- Data on people who buy jewelry: (age,salary (in \$ 150k))

- A(25,60) D (45,60) G(50,75) **J(50,100)**

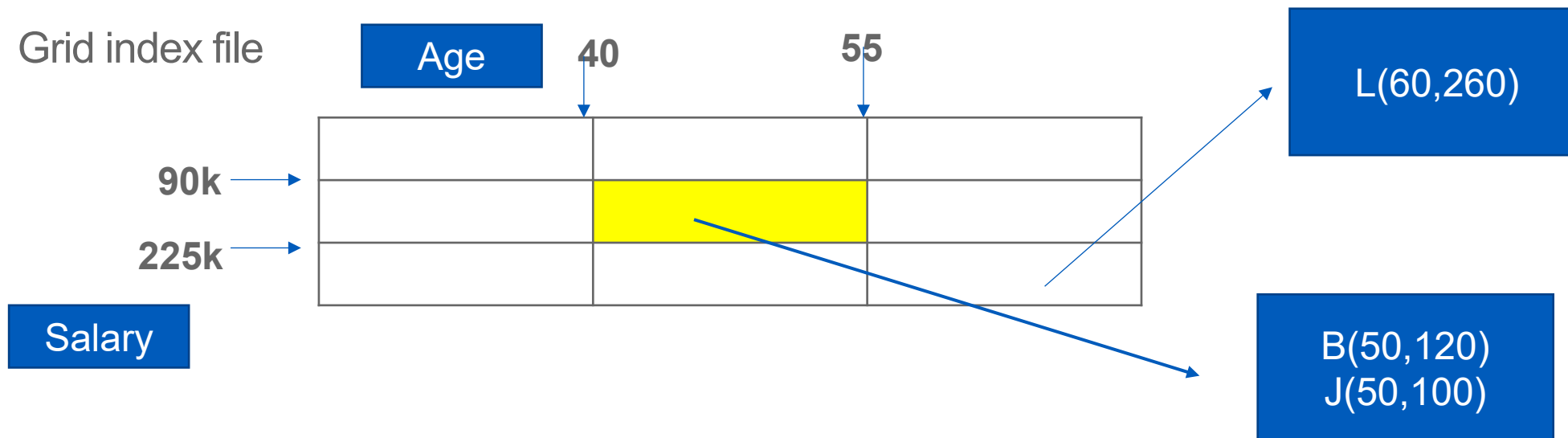
- B(50,120)** E(70,110) H(85,140) K(30,260)

- C(25,400) F(45,350) I(50,275) L(60,260)

- Ranges: **Age** 0-40 40-55 $\geq 55+$

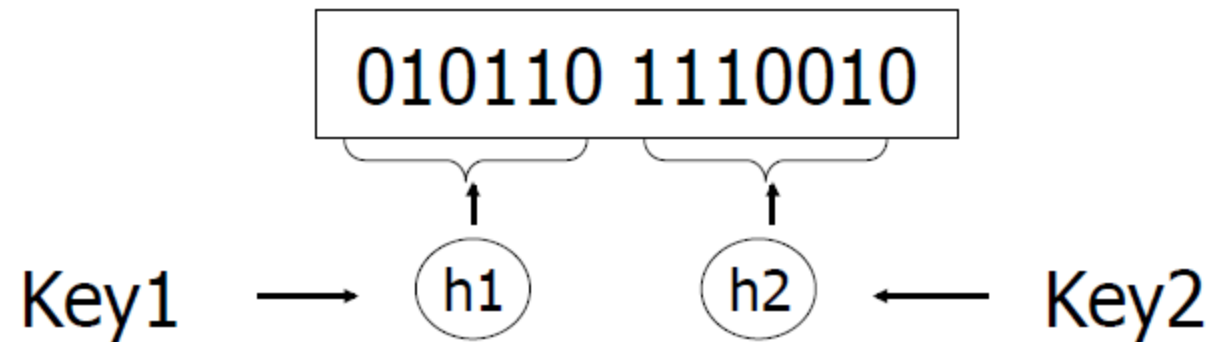
- salary** 0-90k 90k-225k $\geq 225k$

Grid index file



Partitioned Hash Functions

- Basic Idea:



Example

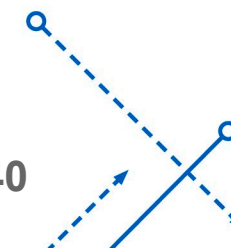
000	
001	<Fred>
010	
011	
100	
101	<Joe> <Sally>
110	
111	

<Fred,toy,10k>

<Joe,sales,10k>

<Sally,art,30k>

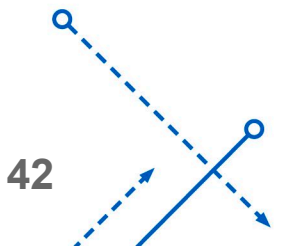
- $h_1(\text{toy}) = 0$
- $h_1(\text{sales}) = 1$
- $h_1(\text{art}) = 1$
- $h_2(10k) = 01$
- $h_2(20k) = 11$
- $h_2(30k) = 01$
- $h_2(40k) = 00$



TRANSACTION

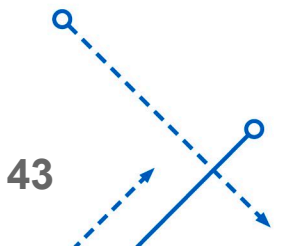
Lec 24-26

- It is motivated by two important independent requirements
 - Concurrent database access
 - Resilience to system failures



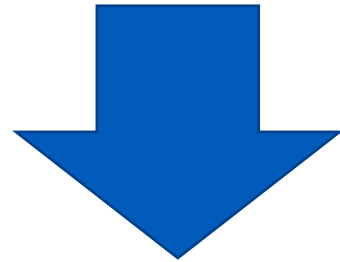
Definition of Transaction and Schedule

- A transaction is the DBMS's abstract view of a user program: a sequence of reads and writes actions.
- A schedule is a list of actions (reading, writing, aborting, or committing) from a set of transactions and the order in which two actions of a transaction T appear in a schedule must be the same as the order in which they appear in T.



ACID Properties

- **Atomicity:** All actions in the transaction happen, or none happen.
- **Consistency:** If each transaction is consistent and the DB starts consistent, then it ends up consistent.
- **Isolation:** Execution of one transaction is isolated from that of other transactions.
- **Durability:** If a transactions commits, its effects persist.

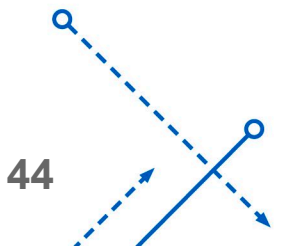


Atomicity: “all or nothing”

Consistency: “it looks correct to me”

Isolation: “as if alone”

Durability: “survive failures”



Concurrency Control

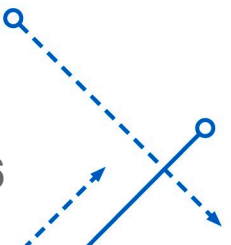
- Multiple concurrent transactions $T_1, T_2, \dots$
- They read/write common elements $A_1, A_2, \dots$
- How can we prevent unwanted interference ?

The SCHEDULER is responsible for that

The general process of assuring that transactions preserve consistency when executing simultaneously is called *concurrency control*.

Serializable Schedule

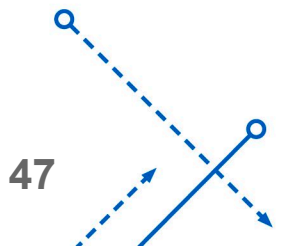
- The role of the *scheduler* is to ensure that the schedule is serializable.
- A schedule that is equivalent to some serial execution of the transactions.
- If each transaction preserves consistency, every serializable schedule preserves consistency
- Transaction = sequence of $r(A)$'s and $w(A)$'s

$$\begin{array}{l} T_1: r_1(A); w_1(A); r_1(B); w_1(B) \\ T_2: r_2(A); w_2(A); r_2(B); w_2(B) \end{array}$$


Conflicting operations

We need a formal notion of equivalence that can be implemented efficiently based on the notion of "conflicting" operations

- Two operations **conflicts** if :
- They are by different transactions,
- They are on the same object and at least one of them is a write.
- $r_i(X)$ & $w_j(X)$ conflict.
- $w_i(X)$ & $w_j(X)$ conflict,
- $r_i(X)$ & $w_j(X)$ conflict



Example

$$r_1(A); w_1(A); r_2(A); \boxed{w_2(A); r_1(B)}; \boxed{w_1(B); r_2(B)}; w_2(B);$$

Not a conflict

conflict

$$r_1(A); w_1(A); \boxed{r_2(A); r_1(B)}; \overline{w_2(A)}; w_1(B); r_2(B); w_2(B);$$

Not a conflict

$$r_1(A); w_1(A); r_1(B); r_2(A); \boxed{w_2(A); w_1(B)}; r_2(B); w_2(B);$$

Not a conflict

$$r_1(A); w_1(A); r_1(B); \boxed{r_2(A); w_1(B)}; w_2(A); r_2(B); w_2(B);$$

Not a conflict

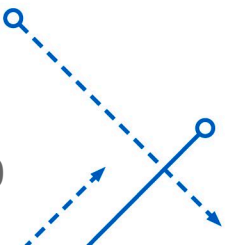
$$r_1(A); w_1(A); r_1(B); w_1(B); r_2(A); w_2(A); r_2(B); w_2(B);$$

Serializability

- Given these conflicts, we now can understand what it means for a schedule to be serializable.
- → This is to check whether schedules are correct.
- **Basic Assumption** – Each transaction preserves database consistency.
- Two form of serializability
 1. **Conflict serializability**
 2. **View serializability**

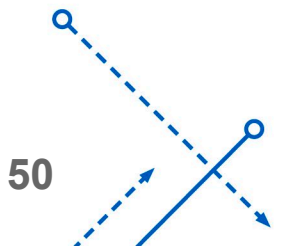
All DBMS
support
this

This is harder but provide
more concurrency



Conflict Serializable Schedules

- Two schedules are **conflict equivalent** if:
 - Involve the same actions of the same transactions
 - Every pair of conflicting actions is ordered the same way
- Schedule S is **conflict serializable** if S is conflict equivalent to some serial schedule



Serializable Schedule

- A schedule is *serializable* if it is equivalent to a serial schedule.
- Is this a serial schedule?
- No
- In this schedule, T_2 acts on A , after T_1 does, but before T_2 acts on B .
- But is it *serializable*?
- Yes
- However, we see that the effect of the two transactions scheduled in this manner is the same as for the serial schedule (T_1, T_2)

T_1	T_2	A	B
		25	25
READ(A, t)			
$t := t + 100$			
WRITE(A, t)		125	
	READ(A, s)		
	$s := s * 2$		
	WRITE(A, s)	250	
READ(B, t)			
$t := t + 100$			
WRITE(B, t)			125
	READ(B, s)		
	$s := s * 2$		
	WRITE(B, s)		250

Figure 18.5: A serializable, but not serial, schedule

Example : Non serializable schedule

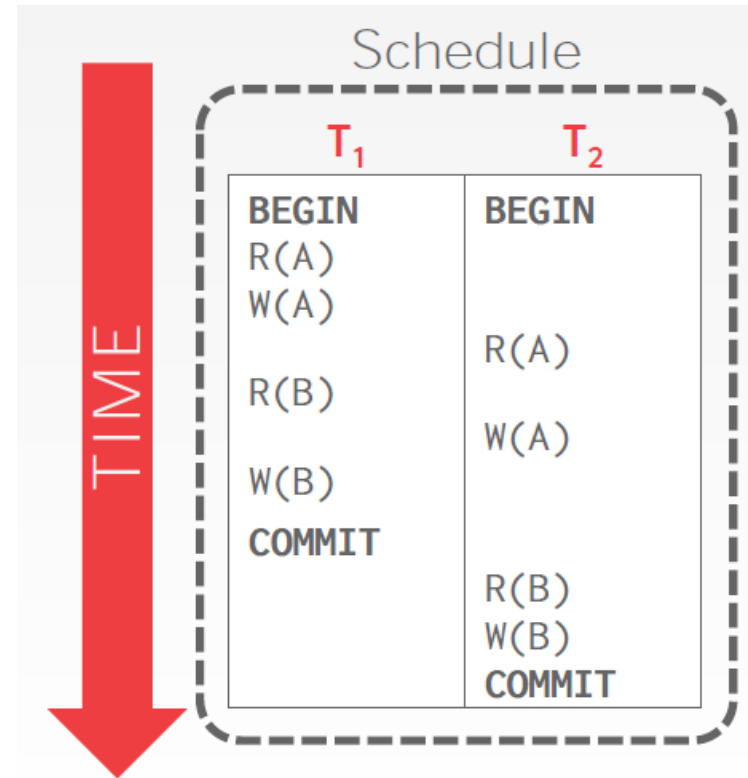
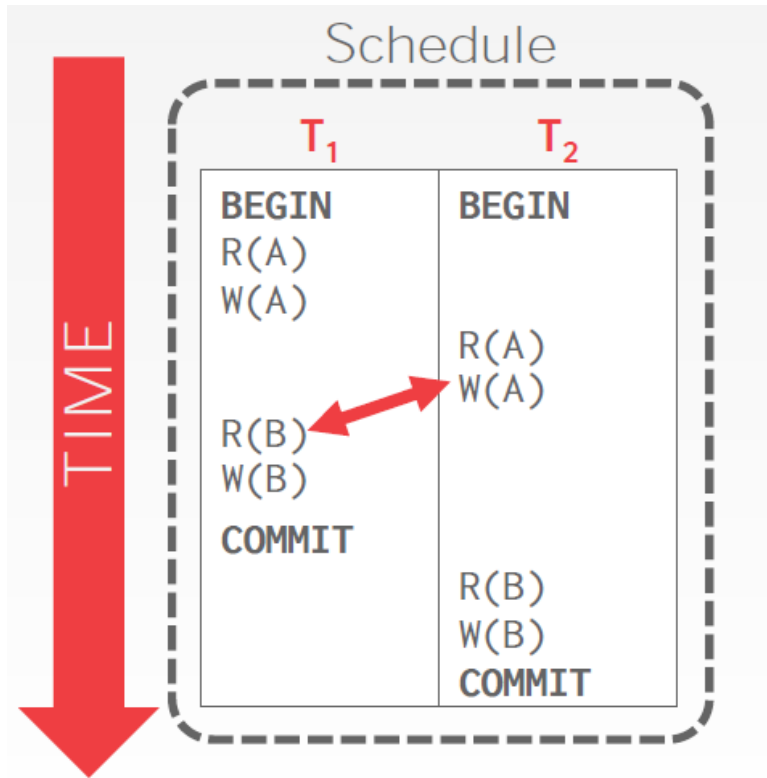
T_1	T_2	A	B
		25	25
READ(A,t)			
t := t+100			
WRITE(A,t)		125	
	READ(A,s)		
	s := s*2		
	WRITE(A,s)	250	
	READ(B,s)		
	s := s*2		
	WRITE(B,s)		50
READ(B,t)			
t := t+100			
WRITE(B,t)			150

- A schedule is *serializable* if it is equivalent to a serial schedule.
- Is this *serializable*?
- *No*, as it takes the consistent state $A = B = 25$ and leaves the database in an inconsistent state, where $A = 250$ and $B = 150$

Figure 18.6: A nonserializable schedule

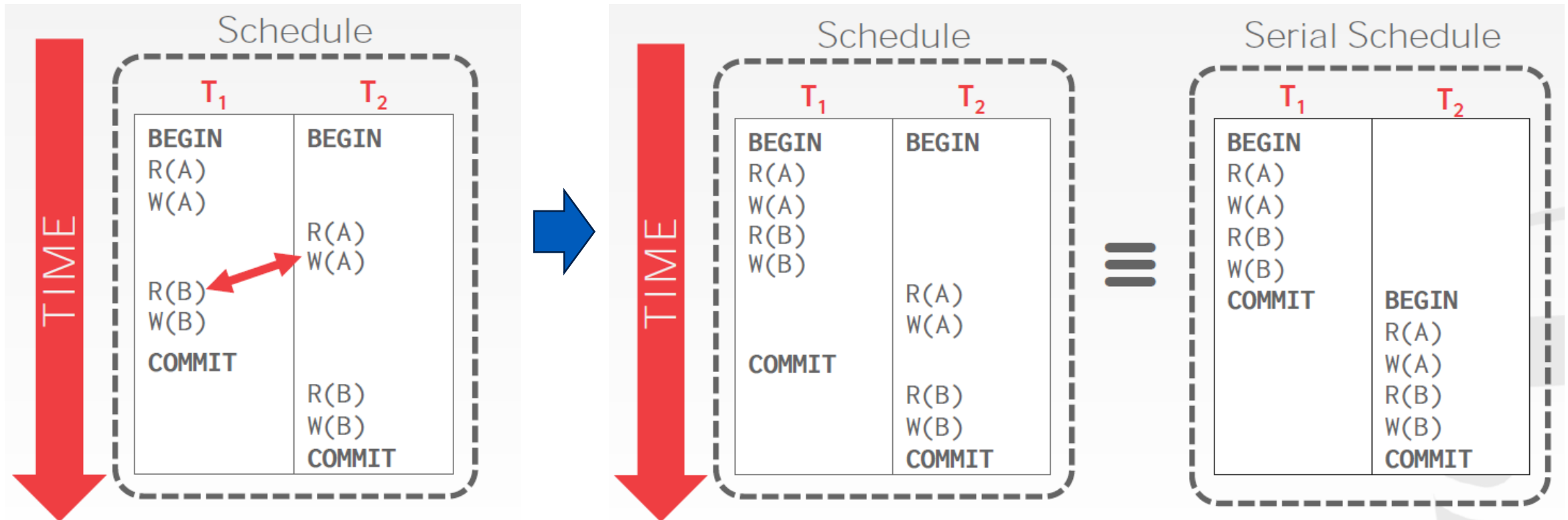
Conflict Serializable

- Schedule S is conflict serializable if you are able to transform S into a serial schedule by swapping consecutive non-conflicting operations of different transactions.



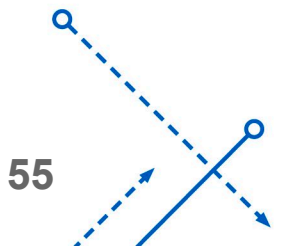
Conflict Serializability

A schedule is *conflict serializable* if it can be transformed into a serial schedule by a series of swapping of adjacent non-conflicting actions



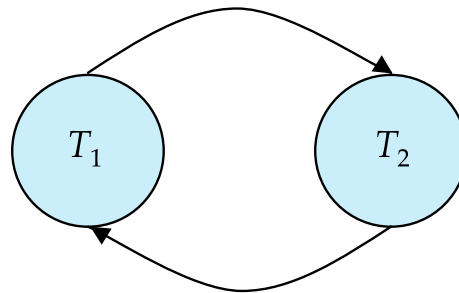
Conflict Serializable

- Swapping operations is easy when there are only two trans in the schedule. It's cumbersome when there are many trans.
- Are there any faster algorithms to figure this out other than transposing operations?



Testing for Serializability

- Consider some schedule of a set of transactions $T_1, T_2, \dots, T_n$
- Precedence graph** — a direct graph where the vertices are the transactions (names).
- We draw an arc from T_i to T_j if the two transaction conflict, and T_i accessed the data item on which the conflict arose earlier.
- We may label the arc by the item that was accessed.
- Example of a precedence graph

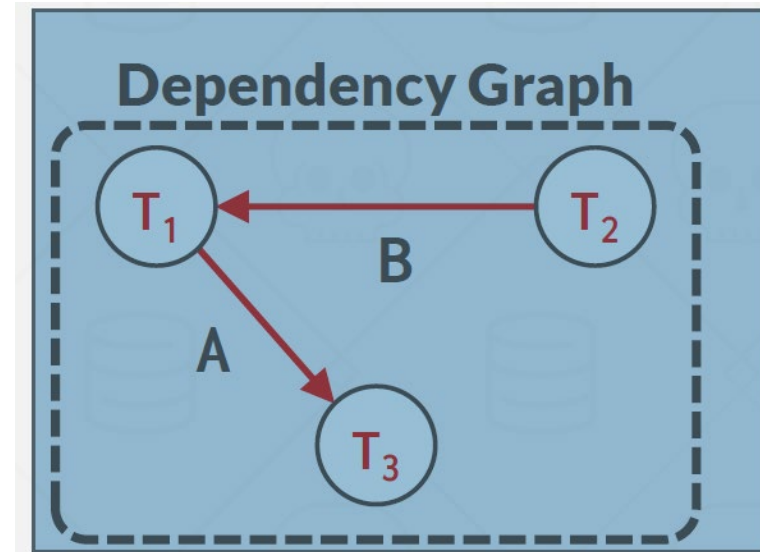


- The schedule is serializable if and only if the precedence graph is acyclic (no loop)



Example

T_1	T_2	T_3
BEGIN R(A) W(A) R(B) W(B) COMMIT	 BEGIN R(B) W(B) COMMIT	 BEGIN R(A) W(A) COMMIT



Example : View Serializability

Schedule 1

T1	T2	T3
R(A)		
	W(A)	
W(A)		
		W(A)



Schedule 1

T1	T2	T3
R(A)		
W(A)		
	W(A)	
		W(A)

View Equivalent

- **View Serializability** allows for (slightly) more schedules than **Conflict Serializability** does.
- → But is difficult to enforce efficiently.
- In practice, **Conflict Serializability** **is** what systems support because it can be enforced efficiently.

